CPSC Compliance as a Service (CaaS) — Technical Brief

Version: 0.1 (Draft)
Date: March 2026
Author: BitConcepts, LLC
Classification: Business Confidential — Pre-Release

Executive Summary

Compliance today is a manual, after-the-fact process: teams write policies in English, implement them as scattered conditional logic, hire auditors to verify they were followed, and discover violations weeks or months later. This approach is expensive ($5.47M average cost of a compliance failure for large enterprises), fragile (human interpretation of policy language), and structurally unable to prevent violations — it can only detect them after the fact.

CPSC Compliance as a Service (CaaS) replaces this model with one where compliance violations are **structurally impossible by construction**. Regulatory controls, governance policies, and security frameworks are expressed as declarative constraints in CAS-YAML — a machine-readable, version-controlled specification format. A deterministic projection engine enforces these constraints on every system state change. No proposal — from a human operator, an AI agent, or an automated workflow — can bypass constraint evaluation. Every decision produces a replayable, auditable transcript.

The same CAS-YAML specification runs on two runtimes: a **software engine** (Rust, cloud or edge) for production deployment, and an **RTL hardware fabric** (VHDL targeting FPGA/ASIC) for line-rate enforcement where microsecond-scale governance is required. Write the policy once; enforce it everywhere.

CaaS is the primary product embodiment of **Constraint-Projected State Computing (CPSC)**, realized through the **CPSC-RE Reasoning Engine** — a layered, deterministic, projection-based system covered by U.S. Provisional Patent Application No. 63/980,251 (filed February 11, 2026).

---

1. The Problem: Compliance Is Broken

1.1 The Current State of Affairs

Modern compliance programs — SOC 2, HIPAA, NIST 800-53, FedRAMP, PCI DSS, GDPR, CMMC — share a common failure mode:

1. **Policies are written in natural language.** A SOC 2 control like "Access to production systems must be restricted to authorized personnel" leaves interpretation to every engineer who implements it.
2. **Implementation is scattered across code.** The "policy" lives as `if` statements, middleware checks, IAM rules, and configuration files distributed across dozens of services. No single artifact captures the complete policy.
3. **Compliance is verified by sampling.** Auditors check logs, interview staff, and inspect configurations at a point in time. Between audits, the system can drift arbitrarily.
4. **Violations are discovered retroactively.** Data breaches, unauthorized access, configuration drift, and policy exceptions surface in post-mortems, not at the moment they occur.
5. **Evidence collection is manual.** Compliance teams spend months assembling screenshots, log exports, and attestation documents for each audit cycle.

1.2 Why This Is Getting Worse

Three accelerating trends make the traditional model unsustainable:

Regulatory volume is expanding. The number of regulatory requirements affecting a typical enterprise has grown 3× in the past decade. New frameworks (EU AI Act, SEC cyber disclosure rules, DORA, NIS2) arrive faster than compliance teams can operationalize them.

System complexity outpaces human review. Cloud-native architectures, microservices, multi-cloud deployments, IaC pipelines, AI/ML models, and agentic automation create a combinatorial explosion of state that no human audit process can exhaustively verify.

AI and agentic systems introduce new governance gaps. When an LLM-driven agent modifies infrastructure, deploys code, or makes authorization decisions, traditional compliance processes — designed for human-in-the-loop approval — have no structural mechanism to enforce policy. The agent acts faster than any approval pipeline can review.

1.3 The Core Structural Problem

The fundamental issue is architectural, not operational:

> ***Traditional compliance systems observe and report. They do not enforce.***

Monitoring tools, SIEM platforms, and GRC software detect that a policy was violated. They cannot prevent the violation from occurring. The system trusts that operators, code, and automation will respect policies. When that trust fails — through misconfiguration, human error, adversarial action, or AI hallucination — the violation has already happened.

What is needed is a system where **violation is structurally impossible**: where the enforcement mechanism is not a check layered on top of the system, but a fundamental property of how state

transitions occur.

---

2. The Solution: Constraint-Projected Compliance

2.1 Core Architecture

CPSC CaaS inverts the compliance model:

```
TRADITIONAL:  Act → Log → Detect → Remediate → Audit → Report
CPSC CaaS:    Propose → Project → Accept/Reject → Commit → Transcript
```

Every change to system state — a configuration update, an access request, a deployment, a resource allocation — is treated as a **proposal**. The CPSC-RE projection engine evaluates the proposal against all declared constraints. If projection converges to a valid state, the change is accepted and committed. If it does not, the change is **deterministically rejected** and the system provides structured feedback identifying which constraints were violated and which variables contributed.

No intermediate step can bypass projection. This is not a policy check that runs in parallel and logs warnings — it is the **mechanism by which state transitions occur**.

2.2 CAS-YAML: Compliance Specifications as Code

Policies are expressed in **CAS-YAML** (Constraint Architecture Specification, v1.0.0), a declarative format that defines:

- **State variables**: Every element of the system being governed — user roles, resource allocations, configuration parameters, access levels, data classifications, network rules
- **Variable roles**: Each variable is explicitly classified as Fixed (immutable constants), External (runtime inputs), Free (degrees of freedom — the actual choices being made), or Derived (fully determined by constraints)
- **Constraints**: Declarative, side-effect-free rules over one or more variables that evaluate to satisfied or violated. Hard constraints define correctness (must always hold). Soft constraints define preferences (minimize or maximize an objective)
- **Projection configuration**: Solver parameters, convergence criteria, numeric precision, epoch scheduling
- **Composition**: Layered file inclusion with explicit merge and override semantics — base policies compose with domain-specific overlays and deployment-specific configurations

A CAS-YAML compliance model is:
- **Machine-executable**: Directly consumed by the CPSC-RE projection engine
- **Human-readable**: Plain YAML, understandable by compliance officers and auditors

- **Version-controlled**: Lives in git alongside the systems it governs, with full change history
- **Portable**: The same model runs on software and hardware engines without modification
- **Composable**: Base frameworks (SOC 2, HIPAA) compose with organization-specific policies via file inclusion

2.3 Example: Access Control as CAS-YAML

```yaml
version: 1.0.0
model_id: access_control_soc2

state:
  variables:
- name: user_role
    type: enum
    domain: [admin, operator, viewer, service_account]
    role: external

    - name: target_resource
        type: enum
        domain: [production_db, staging_db, config_store, audit_log]
        role: external

    - name: access_granted
        type: bool
        domain: [true, false]
        role: free

    - name: requires_mfa
        type: bool
        domain: [true, false]
        role: derived

    - name: mfa_verified
        type: bool
        domain: [true, false]
        role: external

constraints:
- id: prod_admin_only
    type: hard
    expression: "if target_resource == 'production_db' then user_role == 'admin'"
```

```
    description: "SOC 2 CC6.1 — Production access restricted to admins"

    - id: mfa_for_production
        type: hard
        expression: "if target_resource == 'production_db' then requires_mfa == true"
        description: "SOC 2 CC6.1 — MFA required for production access"

    - id: mfa_enforcement
        type: hard
        expression: "if requires_mfa == true then mfa_verified == true"
        description: "MFA must be verified before access is granted"

    - id: audit_log_immutable
        type: hard
        expression: "if target_resource == 'audit_log' then access_granted == false"
        description: "SOC 2 CC7.2 — Audit logs are append-only, no direct access"
```

This is not pseudo-code. This is the actual policy specification consumed by the engine. An operator requesting production database access triggers projection: the engine evaluates all constraints, determines whether any valid state exists where `access_granted == true` given the operator's role and MFA status, and returns a deterministic accept or reject.

2.4 The CPSC-RE Engine: How Enforcement Works

CPSC-RE implements CaaS through a layered architecture:

Layer 0 — CAS-YAML (Semantic System Specification)
The compliance policy: variables, roles, constraints, projection configuration. This is the single source of truth for what is allowed.

Layer 1 — Constraint IR & Lowering
The compiler validates the CAS-YAML model, builds a constraint graph capturing variable dependencies, maps constraints to proto-cell kernels, and emits a canonical binary configuration. This lowering step transforms declarative policy into an executable enforcement plan.

Layer 2 — Proto-Cell Kernel ABI
Seven atomic constraint enforcement units — DomainClamp, Equality, Linear, Clause, Cardinality, Penalty, and Generic — that operate on local variable neighborhoods. Each kernel enforces one constraint type. The kernel ABI is shared between software and hardware implementations.

Layer 3 — Epoch Scheduler
Deterministic four-stage execution cycle: Sense (read current state), Compute (evaluate constraints), Evaluate (compute corrections), Commit (atomically update state). Supports Gauss-Seidel

(sequential) and Jacobi (parallel) projection methods. No state changes occur outside commit boundaries.

Layer 4 — Compiler
End-to-end pipeline from CAS-YAML to executable solver. Automatic kernel type inference maps each constraint to the most efficient kernel. Produces annotated kernel configurations for software execution or hardware synthesis.

Layer 5 — Transcripts
Every projection produces a replayable transcript recording: initial state, constraint evaluations, corrections applied, convergence status, and final accepted/rejected state. Transcripts are the audit trail — deterministic, complete, and machine-verifiable.

---

3. Software and Hardware Duality

3.1 One Spec, Two Runtimes

The defining architectural property of CaaS is that the **same CAS-YAML specification** runs on two runtimes without modification:

Software Engine (Rust)
- Runs on any compute platform: cloud VMs, edge devices, containers, developer workstations
- CPSC-RE compiler processes CAS-YAML, builds constraint graph, executes projection
- Suitable for: GRC platforms, CI/CD pipeline gates, API authorization, batch compliance checks
- Latency: sub-millisecond to milliseconds depending on model complexity
- Deployment: Container image, native binary, shared library, REST/gRPC API
- A Python reference implementation exists for experimentation and rapid prototyping; the Rust engine is the production target

Hardware Engine (VHDL targeting FPGA/ASIC)
- Constraint evaluation units run in parallel in programmable logic or silicon
- CAS-YAML compiles to hardware configuration via the same compiler frontend + VHDL RTL backend
- No instruction execution, no program counter — constraints enforced structurally in gates
- Suitable for: line-rate packet inspection, real-time access control, datacenter admission control, embedded safety systems
- Latency: microseconds (bounded, deterministic)
- Deployment: FPGA bitstream (Xilinx/AMD, Intel/Altera), ASIC IP block

3.2 Why This Matters

Traditional compliance tools are software-only. When enforcement must happen at line rate — network packet filtering, real-time access decisions, safety-critical control — software tools add

latency, introduce scheduling jitter, and create bypass opportunities. Hardware enforcement is not just faster; it is structurally different: the constraint fabric is the enforcement mechanism, not a program running on a general-purpose processor that can be interrupted, deprioritized, or bypassed.

For organizations that need both — software for broad coverage, hardware for critical-path enforcement — CaaS provides a single policy language and a unified compliance posture across both.

3.3 Deployment Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        CAS-YAML Policy Models                            │
│  (SOC 2, HIPAA, NIST 800-53, FedRAMP, CMMC, custom org policies)         │
│  Version-controlled in git · Composable via includes · Auditor-readable│
└──────────────────────────────┬──────────────────────────────────────────┘
                               │
                     ┌─────────┴──────────┐
                     │   CPSC-RE Compiler  │
                     │   (CAS-YAML → IR)   │
                     └─────────┬──────────┘
                               │
              ┌────────────────┴────────────────┐
              ▼                                 ▼
┌──────────────────────┐            ┌──────────────────────┐
│   Software Runtime   │            │   Hardware Runtime   │
│   (Rust Engine)      │            │   (VHDL RTL Fabric)  │
│                      │            │                      │
│  Cloud/edge/CI/CD    │            │  FPGA/ASIC           │
│  API gateway gates   │            │  Line-rate enforce   │
│  Batch policy eval   │            │  Network in-path     │
│  ms-scale latency    │            │  µs-scale latency    │
└──────────┬───────────┘            └──────────┬───────────┘
           │                                   │
           └──────────────┬────────────────────┘
                          ▼
              ┌──────────────────────┐
              │    Transcript Log    │
              │  (Immutable Ledger)  │
              │  Replayable · Signed │
```

```
                    │  Machine-verifiable  │
                    └──────────────────────┘
```

---

4. What Makes CaaS Different

4.1 Enforcement, Not Detection

Property	Traditional GRC/SIEM	CPSC CaaS
When violations are found	After the fact (hours to months)	At proposal time (before commit)
Can violations occur?	Yes — detection is not prevention	No — invalid states cannot be committed
Policy expression	Natural language → ad-hoc code	CAS-YAML (machine-executable + human-readable)
Audit evidence	Manual collection (screenshots, logs)	Automatic transcripts (every decision recorded)
Determinism	Not guaranteed (race conditions, timing)	Guaranteed (same inputs → same output, always)
Hardware enforcement	Not available	Same policy, line-rate FPGA/ASIC enforcement

4.2 Determinism and Reproducibility

CPSC-RE enforces determinism at every level:

- **Same inputs + same constraints + same projection parameters = same output.** Always. Across platforms, across runs, across software and hardware implementations.
- **Numeric precision is declared**, not implicit. Fixed-point, bounded integer, rational, or declared floating precision — specified in the CAS-YAML model.
- **Tie-breaking rules are deterministic.** When multiple valid states exist, the selection rule (lexicographic minimum, minimal norm, etc.) is declared in the model.
- **Failure is deterministic and reproducible.** If projection fails, it fails identically given the same inputs. No flaky policy decisions.

This property is essential for regulatory compliance: an auditor can take a transcript, re-run the projection with the same inputs and model version, and get the same result. Compliance decisions are not opinions — they are mathematical facts under declared constraints.

4.3 Projection Cannot Be Bypassed

In traditional systems, enforcement is a layer: a middleware check, a webhook, a policy engine running alongside the system. These layers can be:
- Disabled during "emergencies"
- Bypassed by direct database access
- Skipped by race conditions or misconfigured routing
- Overridden by administrative escalation

In CPSC CaaS, projection is not a layer — it is the **mechanism by which state transitions occur**. There is no "direct access" that bypasses constraints, because the constraints define the state space itself. A proposed state that violates constraints does not get committed — not because a check catches it, but because no valid projected state exists.

This is the difference between a guardrail and a wall. A guardrail can be climbed over. A wall defines where you can stand.

4.4 Composable Framework Models

CAS-YAML's file inclusion and override semantics enable a layered compliance architecture:

```
includes:
- frameworks/soc2-base.yaml       # SOC 2 Type II baseline
    - frameworks/hipaa-phi.yaml       # HIPAA PHI handling overlay
    - org/acme-corp-policy.yaml       # Organization-specific policies
    - env/production.yaml             # Production-specific overrides
```

- **Base framework models** encode regulatory controls (SOC 2 CC criteria, HIPAA safeguards, NIST 800-53 control families) as CAS-YAML constraints
- **Organization overlays** add company-specific policies (naming conventions, team-based access, custom approval workflows)
- **Environment overrides** specialize for deployment context (production vs. staging, US vs. EU data residency)

Merge semantics are explicit: variables merge by name, constraints merge by ID, projection and execution parameters are overridden by last include. Conflicts fail validation — no silent override.

---

5. Target Markets

5.1 Governance, Risk & Compliance (GRC)

Problem: Enterprises spend $5–15M annually on compliance programs (SOC 2, ISO 27001, PCI DSS, HIPAA). The majority of cost is manual evidence collection, policy interpretation, and audit preparation — not the actual enforcement.

CaaS value:
- Encode compliance frameworks as CAS-YAML models → machine-executable, version-controlled
- Projection produces transcripts that serve as continuous compliance evidence → eliminates manual evidence collection
- Framework updates (new NIST revision, new SOC 2 criteria) are model updates, not engineering projects
- Continuous compliance monitoring, not annual point-in-time audits

Customers: Enterprises subject to SOC 2, ISO 27001, PCI DSS; GRC platform vendors (ServiceNow GRC, Archer, LogicGate); managed compliance providers

5.2 Financial Services & Fintech

Problem: Financial institutions operate under multiple overlapping regulatory frameworks (OCC, SEC, FINRA, GDPR, PSD2, Basel III) with real-time transaction monitoring requirements and severe penalties for violations ($billions in annual fines industry-wide).

CaaS value:
- Transaction authorization as constraint projection: every transaction must satisfy AML, KYC, sanctions, and internal risk constraints before execution
- Hardware enforcement for high-frequency trading compliance: line-rate constraint evaluation on FPGA ensures no trade executes outside declared risk bounds
- Deterministic transcripts satisfy regulatory examination requirements
- Composable framework models handle multi-jurisdiction regulatory overlap

Customers: Banks, broker-dealers, payment processors, crypto exchanges, RegTech platforms

5.3 Healthcare & Life Sciences

Problem: HIPAA, HITRUST, FDA 21 CFR Part 11, and clinical trial regulations create complex access control and data handling requirements. Violations carry both financial penalties and patient safety risks.

CaaS value:
- PHI access controls as hard constraints: projection ensures no access pattern violates minimum-necessary, role-based, and purpose-limitation rules
- Clinical trial data integrity: constraint projection ensures data modification follows declared protocols with complete audit trail
- Medical device software compliance: deterministic enforcement suitable for FDA software

validation
- HITRUST CSF controls as composable CAS-YAML models

Customers: Health systems, EHR vendors, life sciences companies, medical device manufacturers, CROs

5.4 Defense & Intelligence Community

Problem: CMMC, NIST 800-171, ITAR, classified information handling, and cross-domain access control require provably correct enforcement with auditable evidence chains. Manual compliance processes are too slow for mission-tempo operations.

CaaS value:
- Security classification enforcement as hard constraints: projection ensures no data movement violates declared classification boundaries
- Cross-domain transfer policy: every transfer proposal projected against multi-level security constraints
- Hardware enforcement for classified systems: FPGA constraint fabric provides defense-in-depth enforcement that cannot be disabled by software compromise
- Deterministic replay for forensic analysis and insider threat investigation
- CMMC/NIST 800-171 controls as CAS-YAML models with DoD-specific overlays

Customers: Defense contractors (all CMMC levels), intelligence community, DoD program offices (PEO, DISA), cleared facilities

5.5 Critical Infrastructure

Problem: NERC CIP (energy), TSA pipeline security directives, water sector cybersecurity requirements, and industrial control system (ICS) security standards require provable enforcement of operational technology (OT) policies — often on systems where software updates are infrequent and uptime requirements are absolute.

CaaS value:
- SCADA/ICS command validation: every operator command projected against safety and security constraints before execution
- Hardware enforcement at the OT/IT boundary: FPGA-based constraint fabric validates all traffic crossing the IT/OT demarcation
- Deterministic, bounded-latency enforcement suitable for real-time control environments
- Constraint models capture both cybersecurity policy (NERC CIP) and physical safety limits in a unified framework

Customers: Utilities (electric, water, gas), pipeline operators, manufacturing, critical infrastructure operators

5.6 AI Governance & Agentic Systems

Problem: AI systems — including LLMs, autonomous agents, and AI-driven automation — make decisions faster than human review processes can operate. EU AI Act, NIST AI RMF, and emerging AI governance frameworks require provable constraints on AI behavior. No existing system provides structural enforcement.

CaaS value:
- Agent proposals as candidate states: every AI-driven action is a proposal that must project into constraint-satisfying space before execution
- Structural prevention of policy violations: AI cannot hallucinate its way past hard constraints
- Deterministic transcripts provide explainability and auditability for AI decisions
- CPSC-Governed Agentic Development (CGAD) framework provides a complete governance architecture for multi-agent systems
- EU AI Act risk classification and transparency requirements as CAS-YAML constraint models

Customers: AI platform vendors, enterprises deploying AI/LLM agents, AI safety organizations, regulated industries using AI

---

6. Intellectual Property

6.1 Patent Position

U.S. Provisional Patent Application No. 63/980,251
Constraint-Projected State Computing Systems, Semantic System Specification, and Applications
Filed: February 11, 2026 | Inventor: Tristen Kyle Pierson | Assignee: BitConcepts, LLC

The provisional covers:
- CPSC computing model (constraint projection as primary computation mechanism)
- CAS-YAML semantic system specification and lowering to constraint architecture
- Software and hardware (proto-cell fabric + epoch controller) embodiments
- **E-11.3**: Policy and Authorization Enforcement
- **E-11.7**: Constraint-Governed Smart Control Plane for Realms and Resources
- **E-11.20**: CPSC-RE Deterministic Reasoning & Compliance Engine (the canonical CaaS embodiment)
- **E-11.15–11.19**: CPSC-Governed Agentic Development (CGAD) framework
- Deterministic transcripts and audit mechanisms
- Composable constraint model architecture

6.2 IP Moat

The CaaS IP position combines three layers of protection:

1. **Computing paradigm**: Constraint projection as the execution model — not a solver invoked episodically, but the mechanism by which state evolves

2. **Specification format**: CAS-YAML as the portable, composable policy language that compiles to both software and hardware enforcement
3. **Dual-runtime enforcement**: The same policy spec running on software engines and hardware constraint fabrics — competitors would need to replicate both runtimes and the compilation pipeline

6.3 Non-Provisional Conversion

The provisional establishes priority date February 11, 2026. Non-provisional conversion deadline: February 11, 2027. The non-provisional will include specific claims targeting CaaS enforcement, CAS-YAML compilation, hardware constraint fabric enforcement, and transcript generation.

---

7. Technical Feasibility

7.1 Technology Readiness Assessment

Component	Status	TRL
CPSC-RE Compiler (CAS-YAML → Constraint IR)	Implemented (Python ref), tested	4
	Proto-Cell Kernel ABI (7 kernel types)	Specified, Python reference impl
	Epoch Scheduler (Gauss-Seidel/Jacobi)	Implemented (Python ref), tested
	CAS-YAML v1.0.0 specification	Published, stable
	Transcript generation	Specified in Epoch Scheduler
	Software projection engine (Rust)	Architecture defined, in development
	Python reference engine	Implemented (experimental)
	RTL constraint fabric architecture (VHDL)	Specified, partial impl

Component	Status	TRL
	VHDL synthesis from CAS-YAML	Architecture defined, not impl
	Compliance framework models (SOC 2, etc.)	Not yet defined
	REST/gRPC API for CaaS cloud deployment	Not yet implemented

7.2 What Exists Today

A Python reference implementation of the CPSC-RE engine exists for experimentation and validation:
- Full CAS-YAML v1.0.0 parser and validator
- Constraint graph builder with automatic dependency analysis
- Compiler pipeline (CAS-YAML → annotated kernel configurations → projection execution)
- Seven kernel types covering all constraint patterns (domain bounds, equality, linear, clausal, cardinality, penalty, generic)
- Epoch-based projection with convergence detection and failure reporting
- Formal reasoning queries: FEASIBILITY, OPTIMIZATION, VALIDATION

The production software engine is being developed in **Rust** for performance, memory safety, and deployment as a native binary or shared library suitable for cloud, edge, and embedded targets.

The RTL hardware architecture is specified in detail (proto-cell fabric, epoch controller FSM, constraint evaluation units, commit logic) in **VHDL** with partial implementation.

7.3 Development Roadmap

Phase 1 (3 months): CaaS Software Platform
- Rust CPSC-RE engine: core projection engine, CAS-YAML parser, constraint graph, kernel runtime
- Define SOC 2, HIPAA, and NIST 800-53 base framework models in CAS-YAML
- Build REST/gRPC API for cloud-hosted constraint projection
- Implement transcript storage and query interface
- Integrate with CI/CD (GitHub Actions, GitLab CI) as pipeline compliance gate

Phase 2 (3 months): Enterprise Integration
- Connector framework for identity providers (Okta, Azure AD), cloud platforms (AWS, Azure, GCP), and infrastructure tools (Terraform, Kubernetes)
- Dashboard for compliance posture visualization
- Framework model library (PCI DSS, CMMC, ISO 27001, GDPR)
- Multi-tenant SaaS deployment

Phase 3 (6 months): Hardware Enforcement
- Complete VHDL constraint fabric implementation
- CAS-YAML → VHDL → FPGA bitstream compilation pipeline
- Line-rate enforcement demo on Xilinx/AMD FPGA
- Network in-path deployment for packet-level policy enforcement
- Defense/IC evaluation readiness

Phase 4 (6+ months): Market Expansion
- Regulatory certification prep (FedRAMP, StateRAMP)
- CMMC assessment tooling integration
- AI governance and CGAD deployment for agentic systems
- ASIC feasibility study for high-volume embedded enforcement

---

8. Competitive Landscape

8.1 Traditional GRC Platforms (ServiceNow, Archer, OneTrust)

What they do: Workflow management, evidence collection, risk registers, audit coordination.
What they don't do: Enforce compliance at the system level. These are tracking tools, not enforcement engines. A GRC platform records that a control exists and collects evidence that it was followed — it cannot prevent a violation.

CaaS differentiator: CaaS enforces constraints at the point of state change. GRC platforms and CaaS are complementary — CaaS provides the enforcement engine, GRC provides the workflow and reporting layer.

8.2 Policy-as-Code (Open Policy Agent, HashiCorp Sentinel, AWS Cedar)

What they do: Express authorization policies in code (Rego, Sentinel, Cedar), evaluate policy decisions at API gateways and infrastructure tools.
Limitations: Policies are evaluated procedurally — each policy check is an independent function call. No constraint graph, no projection, no convergence semantics. Policies can be inconsistent without detection. No hardware enforcement path. No deterministic transcript with replay guarantees.

CaaS differentiator: CAS-YAML models are constraint systems, not policy functions. The projection engine evaluates all constraints jointly, detects conflicts at compile time, and produces deterministic results. The same model compiles to hardware. Transcripts are replayable proofs, not log entries.

8.3 SIEM / Continuous Monitoring (Splunk, Datadog, CrowdStrike)

What they do: Ingest telemetry, detect anomalies, alert on policy violations.
What they don't do: Prevent violations. By the time a SIEM fires an alert, the violation has already occurred. SIEM is inherently a detection tool.

CaaS differentiator: CaaS prevents violations from being committed. SIEM complements CaaS by monitoring for environmental changes that should trigger constraint model updates.

8.4 Zero Trust / ABAC Platforms (Zscaler, Axiomatics)

What they do: Attribute-based access control, microsegmentation, continuous authentication.
Limitations: Evaluate access decisions based on attribute rules. Rules are evaluated independently — no joint constraint satisfaction, no projection, no formal convergence. No hardware enforcement path for edge/embedded.

CaaS differentiator: CaaS evaluates all constraints jointly through projection. A valid access decision satisfies all constraints simultaneously — not just the individual rules that happened to be checked.

---

9. Business Model

9.1 Revenue Streams

1. **SaaS Platform**: Cloud-hosted CaaS engine with REST/gRPC API. Per-seat or per-projection pricing. Includes dashboard, transcript storage, and framework model library.
2. **Framework Model Licensing**: Curated CAS-YAML models for major compliance frameworks (SOC 2, HIPAA, NIST 800-53, PCI DSS, CMMC). Subscription-based updates as frameworks evolve.
3. **Hardware IP Licensing**: VHDL constraint fabric IP for FPGA/ASIC integration. Per-unit or per-design licensing to defense contractors, networking equipment vendors, and ICS manufacturers.
4. **Professional Services**: Custom CAS-YAML model development for organization-specific policies. Integration consulting. Compliance engineering.
5. **Government Contracts**: SBIR/STTR programs for defense and IC applications. Direct contracts for CaaS deployment in classified environments.

9.2 Market Sizing

Segment	TAM (Annual)	Basis
GRC software	$15–20B	Global GRC market (growing ~14% CAGR)
RegTech / compliance automation	$8–12B	Financial services compliance technology
Defense cybersecurity & compliance	$10–15B	DoD/IC cybersecurity and CMMC compliance
Healthcare compliance technology	$3–5B	HIPAA, HITRUST, FDA compliance tools
Critical infrastructure security	$3–5B	NERC CIP, TSA directives, ICS security
AI governance	$2–4B	Emerging market (EU AI Act, NIST AI RMF)
Total TAM	**$41–61B**	

CaaS is a platform/engine, not a point solution. Realistic SAM is the enforcement-engine and policy-model slice of these markets.

---

10. Risk Analysis

Risk	Severity	Mitigation
Enterprise sales cycle length	High	Start with developer-facing SaaS and CI/CD integration; enterprise sales follow adoption
Regulatory framework modeling complexity	Medium	Start with well-structured frameworks (SOC 2, NIST 800-53); build model library incrementally
Hardware enforcement timeline	Medium	Software-first go-to-market; hardware is a differentiator, not a prerequisite
Competition from policy-as-code incumbents (OPA, etc.)	Medium	CaaS is complementary at the low end (API auth) and differentiated at the high end (joint constraint satisfaction, hardware, transcripts)
Patent prosecution risk	Low–Medium	Provisional establishes priority; multiple independent embodiments provide claim diversification

Risk	Severity	Mitigation
Adoption requires mindset shift (policy-as-constraints)	High	Developer tooling, migration guides, side-by-side OPA comparison, framework model library reduce friction
CAS-YAML modeling errors	Medium	Compiler validates constraint consistency; test projection with known-good and known-bad proposals before deployment

---

11. The Ask

What We're Looking For

1. **Strategic partners** with enterprise compliance platforms to integrate CaaS as the enforcement engine behind their GRC/workflow tooling
2. **Defense/IC partners** with existing ATO'd infrastructure and CMMC assessment experience for classified deployment validation
3. **FPGA/hardware partners** for RTL constraint fabric implementation and line-rate enforcement demo
4. **Seed investment** ($1.5M–$3M) to fund Phase 1–2 development: SaaS platform, framework model library, CI/CD integration, and initial enterprise pilots
5. **Design partners** (2–3 enterprises) willing to run CaaS alongside existing compliance programs for side-by-side validation

What We Bring

- **Patented computing paradigm** — U.S. Provisional 63/980,251 covering CPSC, CAS-YAML, software + hardware enforcement, and transcripts
- **Working CPSC-RE engine** — Python reference implementation (CAS-YAML compiler, constraint graph, 7 kernel types, epoch-based projection, formal reasoning queries); Rust production engine in development
- **Specified hardware architecture** — VHDL proto-cell fabric, epoch controller FSM, constraint evaluation units, deterministic commit logic
- **CAS-YAML v1.0.0** — Published specification for declarative constraint architecture
- **Deep technical foundation** — Comprehensive specification corpus (CPSC Specification, CAS-YAML Specification, CPSC-RE Compiler, Epoch Scheduler, Proto-Cell Kernel ABI, Constraint IR Binary Format, RTL Architecture)

---

12. Summary

Every organization subject to regulatory compliance faces the same structural problem: policies are written in English, implemented as scattered code, verified by sampling, and discovered to be violated after the fact. This model was designed for a world of manual processes, annual audits, and human-speed operations. It is fundamentally inadequate for cloud-native systems, AI-driven automation, and real-time operations.

CPSC CaaS replaces observation-based compliance with **enforcement-by-construction**. Regulatory controls become machine-executable constraints. The projection engine is the enforcement mechanism — not a check layer, but the way state transitions occur. Every decision produces a deterministic, replayable transcript. The same policy runs on software engines for broad coverage and hardware fabrics for line-rate enforcement.

Compliance violations do not get detected. They do not occur.

That is the value proposition.

References

1. CPSC Specification v0.1 — `docs/specification/CPSC-Specification.md`
2. CAS-YAML Specification v1.0.0 — `docs/specification/CAS-YAML-Specification.md`
3. CPSC-RE Compiler — `cpsc-engine-rtl/docs/CPSC-RE-Compiler.md`
4. CPSC-RE Epoch Scheduler — `cpsc-engine-rtl/docs/CPSC-RE-Epoch-Scheduler.md`
5. CPSC-RE Proto-Cell Kernel ABI — `cpsc-engine-rtl/docs/CPSC-RE-Proto-Cell-Kernel-ABI.md`
6. CPSC RTL Architecture — `cpsc-engine-rtl/docs/CPSC-RTL-ARCHITECTURE.md`
7. U.S. Provisional Patent Application No. 63/980,251, *Constraint-Projected State Computing Systems, Semantic System Specification, and Applications*, filed February 11, 2026
8. CPSC Embodiments Overview — `docs/public/cpsc-embodiments-overview.md`

Last updated: March 2026
Maintainer: BitConcepts, LLC

CPSC-CaaS-Brief.md | © 2026 BitConcepts, LLC | Business Confidential